

Módulo V

Backend

Node.js

Node JS

Javascript nació como un lenguaje de programación para el navegador, que servía de apoyo a las páginas HTML+CSS para dotar de interactividad y de mayor dinamismo a su funcionamiento. Con el tiempo, se comienza a utilizar más y más Javascript en los sitios web, luego se hace posible utilizar Javascript fuera del navegador a través de Node.js.

Node.js es un entorno de ejecución de JavaScript del lado del servidor, fué escrito en C++ con el intérprete de Google Chrome V8 extendido. Es decir, que utiliza un intérprete de Javascript y lo extiende para agregarle bastantes características propias de un servidor.



JavaScript del lado del servidor



JavaScript no fue pensado originalmente para trabajar del lado del servidor, de tal forma que en pro de lograr que pudiese ser un lenguaje que se pudiera utilizar para escribir programación del lado del servidor se tuvieron que agregar nuevas características a V8 en Nodejs.

- Organización del código con piezas reutilizables (módulos).
- Manejo de archivos (filesystem).
- Manejo de bases de datos.
- Interacción con web services.
- Interacción mediante el protocolo HTTP.
- Manejo de procesos que puedan tomar mucho tiempo en ser ejecutados.



Shell de Nodejs



La shell de Nodejs permite ejecutar comandos de Nodejs directamente desde un cliente de Nodejs en la línea de comandos.

Para ingresar a la shell (terminal) de node se debe tipear:

```
node
```

A screenshot of a Windows PowerShell terminal window. The title bar says "Windows PowerShell". The text inside the terminal shows the standard PowerShell startup messages, followed by the command 'node' being entered. The terminal then displays the Node.js v18.16.0 welcome message and the prompt '>'. The user has entered '1+1' and the terminal has outputted '2'.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras.
https://aka.ms/PSWindows

PS C:\Users\lucas> node
Welcome to Node.js v18.16.0.
Type ".help" for more information.
> 1+1
2
> |
```

Shell de Nodejs



Por default las operaciones que regresan un resultado son impresas directamente en la shell, no es necesario imprimirlas, la primera reacción sería entonces almacenar el resultado en una variable así... $total = 1 + 1$, pero al teclear esto sobre la shell también imprime el resultado.

Si no quiero regresar un valor a impresión y solo quiero mantenerlo en una variable, tengo que especificarle que solo quiero que se mantenga el valor en la variable.

```
> let total = 1+1
undefined
> total
2
> |
```

Para salir de la terminal basta con presionar CTRL+C+C.

```
>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
> |
```

NPM

Es el sistema de gestión de paquetes por defecto para Node.js. Permite crear los archivos necesarios para gestionar los desarrollos con Node o JavaScript en general, generando un archivo llamado **package.json** con metadatos del proyecto.

NPM es el registro de software más grande del mundo. Los desarrolladores de código abierto de todos los continentes utilizan npm para compartir y tomar prestados paquetes, y muchas organizaciones también lo utilizan para gestionar el desarrollo privado.

Consta de tres componentes distintos:

- El sitio web
- La interfaz de línea de comandos (CLI)
- El registro



Archivo package.json



Nuestro proyecto posee un archivo donde se definen las dependencias y alguna información adicional, vamos a crear este archivo utilizando el wizard.

```
C:\Users\Your Name> npm init
```

- campo **name**: (nombre) del paquete tenemos que utilizar algo que sea url-friendly (url amigable), vamos a ingresar mi-app.
- Por default, se generará la primer versión de nuestra app, que en este caso será la **1.0.0**.
- campo **description**: (descripción) vamos a escribir Primera aplicación utilizando npm.
- campo **entry point**: (punto de entrada) vamos a especificar el archivo que sirve como punto de entrada a la aplicación, en nuestro caso app.js.
- Continúa con los valores por default, hasta que salga del wizard y regrese el cursor de la línea de comandos.

Instalando dependencia npm

Cuando queremos incorporar una librería externa, lo podemos hacer con el comando

```
npm install LIBRERÍA
```

Ejemplo:

```
npm install moment
```

```
{  
  // ....  
  "dependencies": {  
    "moment": "^2.30.0"  
  }  
}
```

- ^ (caret) quiere decir que se actualice automáticamente todo lo que sea menor o patch, es decir cualquier versión después del primer punto.
 - ^3.9.2 3.*.* retrocompatibilidad, refactorización grande, bug
- ~ (virgulilla) actualizará únicamente los patches (parches) serán actualizados.
 - ~3.9.2 3.9.* bug fix



Instalando proyecto descargado



Cuando descargamos un proyecto, únicamente se descarga el código fuente del mismo, y para poder descargar todas las dependencias debemos ejecutar:

```
npm install | npm i
```

 package-lock.json package.json

Iniciando proyecto con dependencias



creamos el archivo ejemploMoment.js y luego lo ejecutamos con `node - - watch`

```
var moment = require('moment');
moment.locale('es');
moment.weekdays(true).forEach(function(dia) {
  console.log(dia);
});

const fechaCualquiera = moment('2024-08-20');
const nuevaFecha = fechaCualquiera.clone().add(7, 'days');
console.log(`Fecha con 7 días agregados:
${nuevaFecha.format('YYYY-MM-DD')}`);
```

Dependencias globales



Si al comando de install, le agregamos **-g** estamos indicando a npm que la dependencia en lugar de guardarse localmente en el proyecto/node_modules, los guarda en una ruta global del sistema y podría reutilizarse en varios proyectos

Instalamos jshint

```
npm install -g nodemon
```

Dependencias de desarrollo



Existen otro tipo de dependencias que solo son necesarias cuando se desarrolla, pero no son dependencias de nuestra aplicación. Estas dependencias de desarrollo se les llama **development dependencies**. Supongamos que deseamos utilizar jest para ejecutar pruebas mientras se desarrolla una aplicación o un.

```
npm install jest --save-dev || npm install jest -D
```

Nuestro package.json agrega jest como dependencia DEV.

```
"devDependencies": {  
  "jest": "^29.7.0"  
}
```

Nodemon

Nodemon permite analizar proyectos en nodejs, de manera que si se ejecuta un cambio en alguno de los archivos, nodemon cierra y abre nuevamente automáticamente la aplicación.

```
C:\Users\Your Name> npm install -g nodemon
```

guardamos en app.js :

```
var http = require('http');  
var miServidor = http.createServer((peticion, respuesta) => {  
  respuesta.writeHead(200, { 'Content-Type' : 'text/plain' });  
  respuesta.end('Hola mundo!');  
});  
  
miServidor.listen(8080, '127.0.0.1');
```

```
C:\Users\Your Name> nodemon app.js
```



Módulos en NodeJS



Un módulo es un bloque de código reutilizable cuya existencia no impacta de manera accidental otros bloques de código.

Javascript, al menos antes de ES6 no tenía una forma estandarizada de organizar el código de manera que un bloque de código no impactará en otro. Existían formas prevenirlo pero no propiamente como parte del lenguaje.

CommonJS es una forma estandarizada en la forma en la que los módulos deben de ser estructurados. Es decir la forma en la que organizamos el código Javascript. Preparados para trabajar fuera del navegador web

Módulos en NodeJS



Creamos dos archivos, uno llamado index y otro llamado funciones

CommonJS module exports

```
//operaciones.js
function saludar(name) {
    return 'hola ' + name;
}
module.exports = saludar;
```

CommonJS require module

```
//index.js
const miFuncion = require('./operaciones')
console.log(miFuncion('Pepe'));
```

CommonJS module exports

```
//operaciones.js
function saludar(name) {
    return 'hola ' + name;
}
module.exports = { saludar };
```

CommonJS require module

```
//index.js
const { saludar } = require('./operaciones')
console.log(saludar('Pepe'));
```

ES Modules



- .js → por defecto utiliza CommonJS
- .mjs → para utilizar ES Modules
- .cjs → para utilizar CommonJS

ES Modules module exports

```
//funciones.mjs
export function saludar(name) {
  return 'hola ' + name;
}

export function despedir(name) {
  return 'adios ' + name;
}
```

ES Modules import module

```
//index.mjs
import { saludar, despedir } from './operaciones.mjs'
console.log(saludar('Pepe'));
console.log(despedir('Pepe'));
```

Export vs Export default



export default	export (nombrado)
se utiliza cuando el módulo tenga una sola funcionalidad principal.	se utiliza cuando el módulo tenga múltiples exportaciones que deberían ser utilizadas de manera diferenciada.
Solo puede haber una por archivo.	Puede haber múltiples exportaciones.
Se importa sin usar llaves {}.	Se importa usando llaves { }.
Puedes darle cualquier nombre al importar (no necesita coincidir con el nombre en el archivo de origen).	El nombre de la importación debe coincidir exactamente con el nombre exportado.

First class functions



Javascript permite el uso de funciones de primer clase (First Class Functions), es decir que podemos asignar una función a una variable.

```
var sumar = function(x, y){  
    console.log(x + y);  
}  
sumar(4, 5);
```

También se pueden pasar como argumentos a otras funciones, así como ser el resultado de otra función, lo que permite la creación de funciones de orden superior.

```
var sumar = function (x, y) {  
    console.log(x + y);  
}  
let calculadora = function (algunCalculo, x, y) {  
    algunCalculo(x, y);  
}  
calculadora(sumar, 4, 5);
```



Función Flecha (Arrow Function)



Las funciones de flecha nos permiten escribir una sintaxis de función más corta.

```
hello = function() {  
  return "Hello World!";  
}
```

```
hello = () => {  
  return "Hello World!";  
}
```

¡Se hace más corto! Si la función tiene solo una declaración y la declaración devuelve un valor, puede quitar los **corchetes** y la palabra clave **return**

```
hello = () => "Hello World!";
```

Si tiene parámetros, los pasa dentro de los paréntesis:

```
hello = (val) => "Hello " + val;
```

Si solo tiene un parámetro, también puede omitir los paréntesis:

```
hello = val => "Hello " + val;
```



Programación Asíncrona



En la programación asíncrona se refiere a la ejecución de procesos ejecutándose de manera simultánea.

Javascript es síncrono pero **Nodejs es asíncrono**, V8 el motor de JS que utiliza node, convierte el código JS en lenguaje máquina que puede ser interpretado por el CPU.

Callbacks

Un callback es una función que es enviada a otra función para que sea ejecutada en determinado momento.

```
function a(b){  
  b();  
}  
  
a(function(){  
  console.log('Hola mundo!')  
});
```



setTimeout

En una API asíncrona, la API comenzará una operación e inmediatamente retornará (antes de que la operación se complete). Una vez que la operación finalice, la API usará algún mecanismo para realizar operaciones adicionales.

```
setTimeout(function() {  
  console.log('Primero');  
}, 3000);  
console.log('Segundo');
```

```
let tiempo = 0;  
reloj = function(){  
  setTimeout(() => {  
    console.log(tiempo);  
    tiempo += 1;  
    reloj();  
  }, 1000);  
}  
reloj();
```



Filesystem (sistema de archivos)



1. Creamos nuestros archivos.js
2. Inicializamos el módulo fs (filesystem).

```
var fs = require('fs');  
var texto = fs.readFileSync(__dirname + '/archivo.txt', 'utf8');  
console.log(texto);  
console.log("Lectura Síncrona terminada!");
```

3. Vamos a cambiar un poco el código, para hacerlo asíncrono.

```
fs.readFile(__dirname + "/archivo.txt", "utf8", (err, data) => {  
  if (err) {  
    console.error(err);  
    return;  
  }  
  console.log(data);  
});  
console.log("Lectura Asíncrona terminada!");
```

NodeJS input



El input nos permite leer datos de la consola.

```
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question('cual es tu nombre ? ', function (name) {
  rl.question('donde tu vives ? ', function (country) {
    console.log(`${name}, vive en ${country}`);
    rl.close();
  });
});

rl.on('close', function () {
  console.log('\nBYE BYE !!!');
  process.exit(0);
});
```

Servidores HTTP

Arquitectura



Arquitectura de software: Conjunto de decisiones estructurales sobre el diseño y organización de un sistema de software. Define los componentes del sistema y cómo interactúan para cumplir con los objetivos técnicos y de negocio, incluyendo los patrones y principios que guían el desarrollo, mantenimiento y evolución del sistema.

Arquitectura de TI: Diseño global de la infraestructura tecnológica de una organización, incluyendo hardware, software, redes, bases de datos y políticas de seguridad.

Arquitectura de red: Describen cómo se organizan y comunican los dispositivos y aplicaciones dentro de una red. Busca garantizar una comunicación eficiente, segura y escalable entre los distintos sistemas conectados



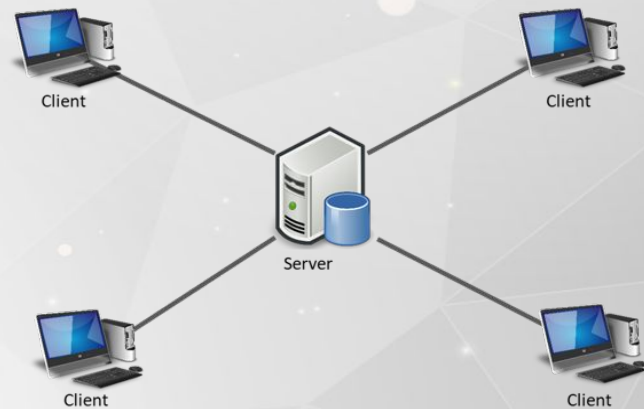
Se refiere a la organización y estructura que define cómo interactúan los componentes principales de un sistema

Arquitectura cliente-servidor



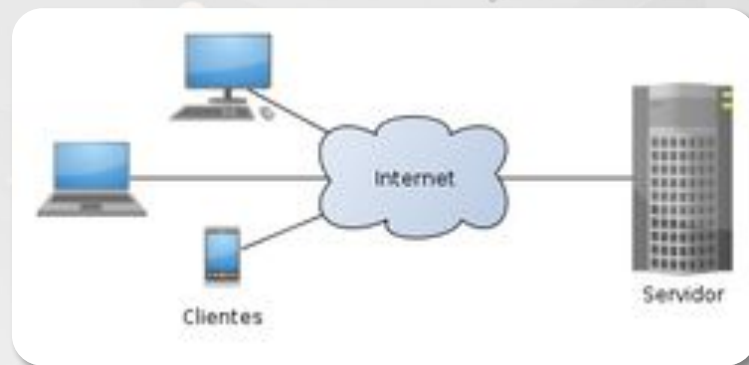
Especifica cómo los clientes (dispositivos o aplicaciones que solicitan servicios) se comunican con los servidores (sistemas que proporcionan estos servicios), estableciendo reglas claras para la transmisión de datos, manejo de solicitudes y respuestas, distribución de cargas y la seguridad. En este sentido, la arquitectura cliente-servidor describe el modelo de diseño que permite una interacción estructurada y eficiente entre ambos roles, asegurando que el sistema funcione de manera coherente y escalable.

En una arquitectura Cliente-Servidor existe un servidor y múltiples clientes que se conectan al servidor para recuperar todos los recursos necesarios para funcionar, en este sentido, el cliente solo es una capa para representar los datos y se detonan acciones para modificar el estado del servidor, mientras que el servidor es el que hace todo el trabajo pesado.



Arquitectura cliente-servidor

En un ecosistema web el almacenamiento y procesamiento de los datos se efectúa en el servidor, la visualización de estos se disponibiliza en los clientes a través de los navegadores web.



Protocolo HTTP



HTTP, de sus siglas en inglés: "Hypertext Transfer Protocol", es el nombre de un protocolo el cual nos permite realizar una petición de datos y recursos, como pueden ser documentos HTML. Es la base de cualquier intercambio de datos en la Web, y un protocolo de estructura cliente-servidor, esto quiere decir que una petición de datos es iniciada por el elemento que recibirá los datos (el cliente), normalmente un navegador Web. Así, una página web completa resulta de la unión de distintos sub-documentos recibidos, como, por ejemplo: un documento que especifique el estilo de maquetación de la página web (CSS), el texto, las imágenes, vídeos, scripts, etc...



Protocolo HTTP Cliente



El agente del usuario, es cualquier herramienta que actúe en representación del usuario. Generalmente esta función es realizada por un navegador Web.

El navegador es siempre el que inicia una comunicación (petición), el servidor nunca la comienza.

Para poder mostrar una página Web, el navegador envía una petición de documento HTML al servidor. Entonces procesa este documento, y envía más peticiones para solicitar scripts, hojas de estilo (CSS), y otros datos que necesite (normalmente vídeos y/o imágenes). El navegador, une todos estos documentos y datos, y compone el resultado final: la página Web. Los javascripts, los ejecuta también el navegador, y también pueden generar más peticiones de datos en el tiempo, y el navegador, gestionará y actualizará la página Web en consecuencia.

Protocolo HTTP Servidor



Al otro lado del canal de comunicación, está el servidor, el cual "sirve" los datos que ha pedido el cliente. Un servidor conceptualmente es una única entidad, aunque puede estar formado por varios elementos, que se reparten la carga de peticiones, u otros programas, que gestionan otros computadores (como caché, bases de datos, servidores de correo electrónico, ...), y que generan parte o todo el documento que ha sido pedido.

Un servidor no tiene que ser necesariamente un único equipo físico, aunque sí que varios servidores pueden estar funcionando en un único computador.

HTTP métodos

Los métodos HTTP permiten comunicar al servidor, mediante una URL, que acción se quiere realizar con un recurso. Los métodos más importantes de HTTP (especialmente para hacer aplicaciones REST) son POST, GET, PUT, DELETE y HEAD.

- Alta = Create,
- Consulta = Read
- Modificación = Update
- Baja = Delete

GET —————> **Leer**

POST —————> **Crear**

PUT —————> **Modificar**

DELETE —————> **Eliminar**



HTTP (GET)



El método GET se emplea para leer una representación de un resource. En caso de respuesta positiva (**200** OK), GET devuelve la representación en un formato concreto: HTML, XML, JSON o imágenes, JavaScript, CSS, etc. En caso de respuesta negativa devuelve **404** (not found) o **400** (bad request). Por ejemplo en la carga de una página web, primero se carga la url solicitada:

Las peticiones que usan el método GET sólo deben recuperar datos.

HTTP: POST / PUT y DELETE

Aunque se puedan enviar datos a través del método GET, en muchos casos se utiliza **POST** O **PUT** por las limitaciones de GET. En caso de respuesta positiva devuelve 201 (created).

Los **POST** requests se envían normalmente con formularios. **PUT** es utilizado normalmente para actualizar contenidos, aunque también puede crearlos. A diferencia de POST es idempotente, si se crea o edita un resource con PUT y se hace el mismo request otra vez, el resource todavía está ahí y mantiene el mismo estado que en la primera llamada.

DELETE simplemente elimina un recurso identificado en la URL. Si se elimina correctamente devuelve 200 junto con un body response, o 204 sin body. DELETE, al igual que PUT y GET, también es idempotente.



Status Code

Un código de estado, o códigos de error, como su propio nombre describe son una serie de códigos de tres cifras estandarizados y que están relacionados con una serie de determinados errores que pueden suceder al introducir en nuestro navegador una dirección web.

Es un mensaje que envía el servidor de un sitio web al navegador para indicar si se pudo cumplir una solicitud. Estos se incluyen en el encabezado HTTP de una página.

Los códigos de estado se agrupan en cinco clases:

- 100–199: Respuestas informativas
- 200–299: Respuestas satisfactorias
- 300–399: Redirecciones
- 400–499: Errores de los clientes
- 500–599: Errores de los servidores



¿Qué es una API?



Una **API** (del inglés, *application programming interface*, en español, *interfaz de programación de aplicaciones*), es un intermediario entre dos sistemas, que permite que una aplicación se comuniquen con otra y pida datos o acciones específicas. Es una abstracción de funciones y procedimientos, básicamente un conjunto de llamadas a ciertas bibliotecas que ofrecen acceso a ciertos servicios y procesos en la programación.

Define las reglas que debe seguir para comunicarse con otros sistemas de software. Los desarrolladores exponen o crean API para que otras aplicaciones puedan comunicarse con sus aplicaciones de manera programática. Por ejemplo, una aplicación de planilla de horas expone una API que solicita el nombre completo de un empleado y un rango de fechas. Cuando recibe esta información, procesa internamente la planilla de horas del empleado y devuelve la cantidad de horas trabajadas en ese rango de fechas.



¿Qué es REST?

REST (del inglés Representational State Transfer) o Transferencia de Estado Representacional es cualquier interfaz entre sistemas que use HTTP para obtener datos o generar operaciones sobre esos datos en todos los formatos posibles, como XML y JSON. No es un protocolo, sino un estilo de arquitectura software que surgió como alternativa a SOAP.

¿Un protocolo cliente/servidor sin estado?

Cada mensaje HTTP contiene toda la información necesaria para comprender la petición, con un conjunto de operaciones bien definidas (POST, GET, PUT y DELETE) y una sintaxis universal para identificar cada recurso (elementos de información) el cual es direccionable únicamente a través de su URI (Identificador Uniforme de Recurso).



Principios REST



- **Interfaz uniforme:** indica que el servidor transfiere información en un formato estándar. El recurso formateado se denomina representación en REST y puede ser diferente de la representación interna del recurso en el servidor. Por ej., el servidor puede almacenar datos como texto pero enviarlos en un formato JSON.
- **Tecnología sin estado:** Cada vez que el cliente pide algo al servidor, el servidor responde sin recordar solicitudes anteriores. Cada petición es independiente.
- **Sistema por capas:** El cliente puede interactuar con intermediarios (como seguridad o lógica de negocios) entre él y el servidor, pero sigue recibiendo respuestas del servidor principal. El cliente no ve estas capas.
- **Almacenamiento en caché:** El servidor puede guardar respuestas en la memoria del cliente para que no tenga que enviar la misma información varias veces, mejorando la velocidad.
- **Código bajo demanda:** El servidor puede enviar código al cliente para personalizar su funcionalidad (ej. cuando un formulario avisa si hay errores mientras lo completas).

RESTful API



Los desarrolladores pueden diseñar API utilizando varias arquitecturas diferentes. Las API que siguen el estilo arquitectónico REST se denominan API REST. Los servicios web que implementan la arquitectura REST se denominan servicios web RESTful. El término API RESTful generalmente se refiere a las API web RESTful. Sin embargo, puede utilizar los términos API REST y API RESTful indistintamente.

Una API REST (también denominada API RESTful o API web RESTful) es una interfaz de programación de aplicaciones (API) que se ajusta a los principios de diseño del estilo arquitectónico de transferencia de estado representacional (REST).



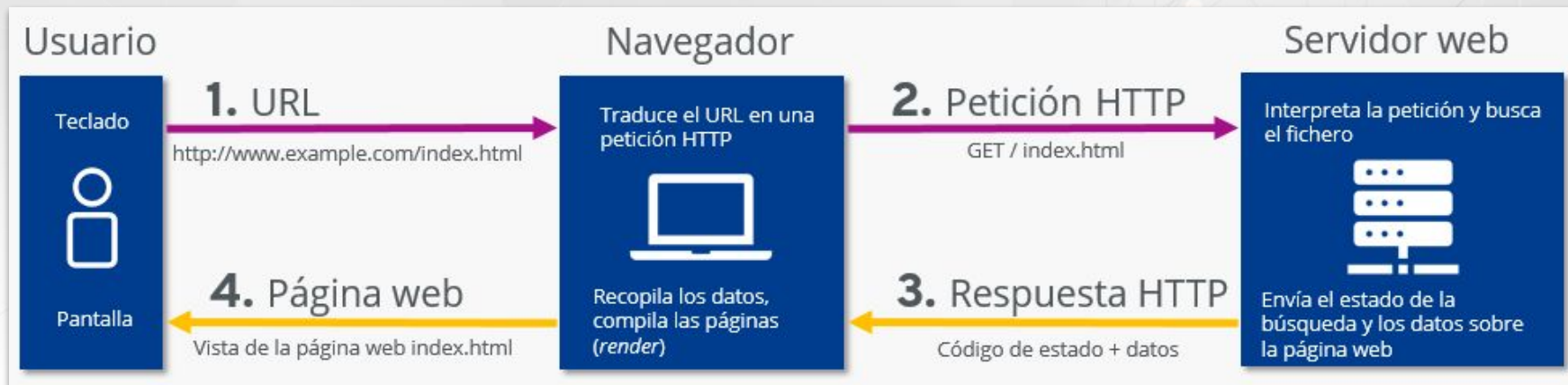
¿Cómo funcionan las API RESTful?



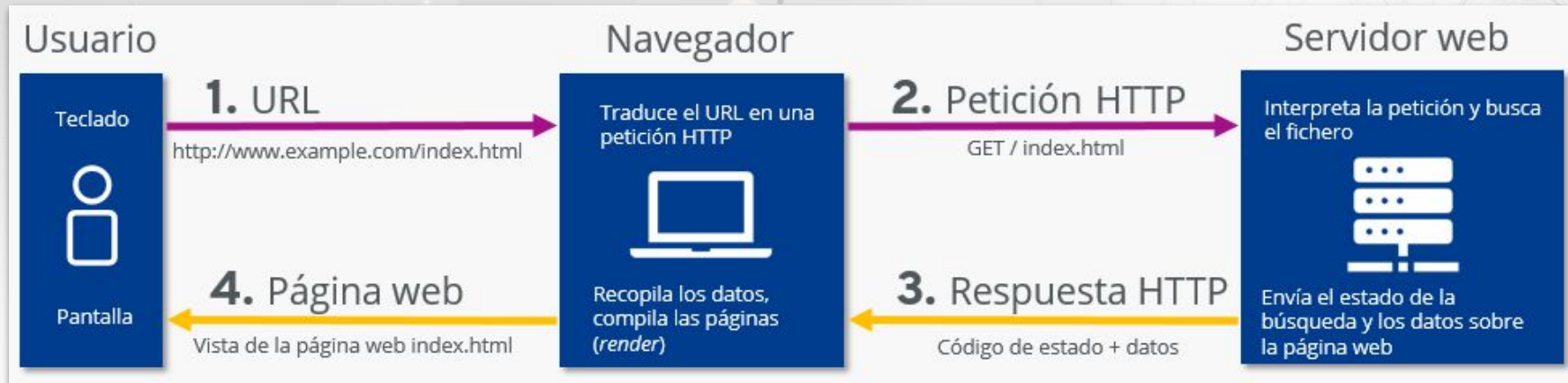
Su función básica es la misma que la de navegar por Internet. El cliente se comunica con el servidor mediante la API cuando necesita un recurso. Los pasos generales para cualquier llamada a la API REST son:

- El cliente envía una solicitud al servidor. El cliente sigue la documentación de la API para dar formato a la solicitud de forma que el servidor la comprenda.
- El servidor autentica al cliente y confirma que éste tiene derecho a realizar esa solicitud.
- El servidor recibe la solicitud y la procesa internamente.
- El servidor devuelve una respuesta al cliente. La respuesta contiene información que indica al cliente si la solicitud se realizó correctamente. La respuesta también incluye cualquier información que haya solicitado el cliente.

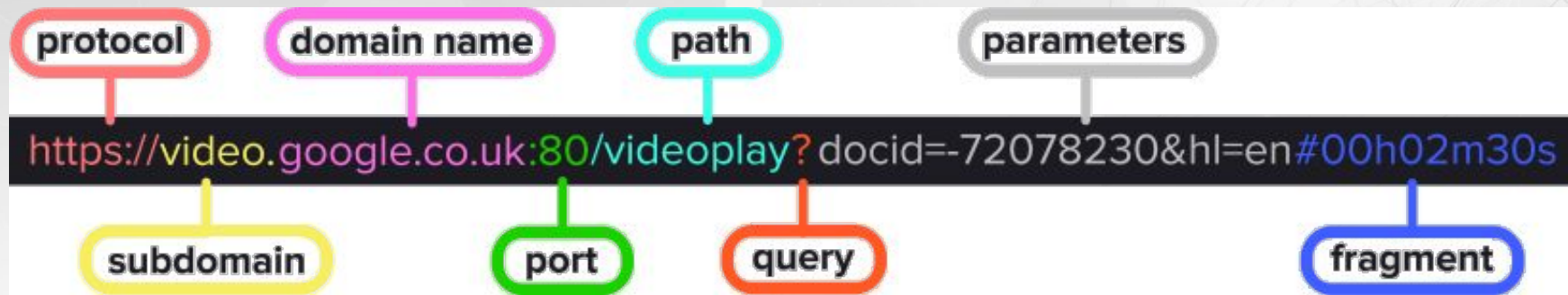
Circuito web



URL



URL



URL

Guardamos en app.js :

```
var url = require('url');  
var adr =  
'http://localhost:8080/default.htm?year=2017&month=february#segundaparte';  
var q = url.parse(adr, true);  
  
console.log(q.protocol);  
console.log(q.hostname);  
console.log(q.port);  
console.log(q.pathname);  
console.log(q.search);  
console.log(q.hash);  
var qdata = q.query;  
console.log(qdata.month);
```



Express

Express es un framework de muy reducido tamaño pero bastante flexible que provee una serie de funcionalidades listas para implementar en aplicaciones web o móviles.



¿Por Qué Express?



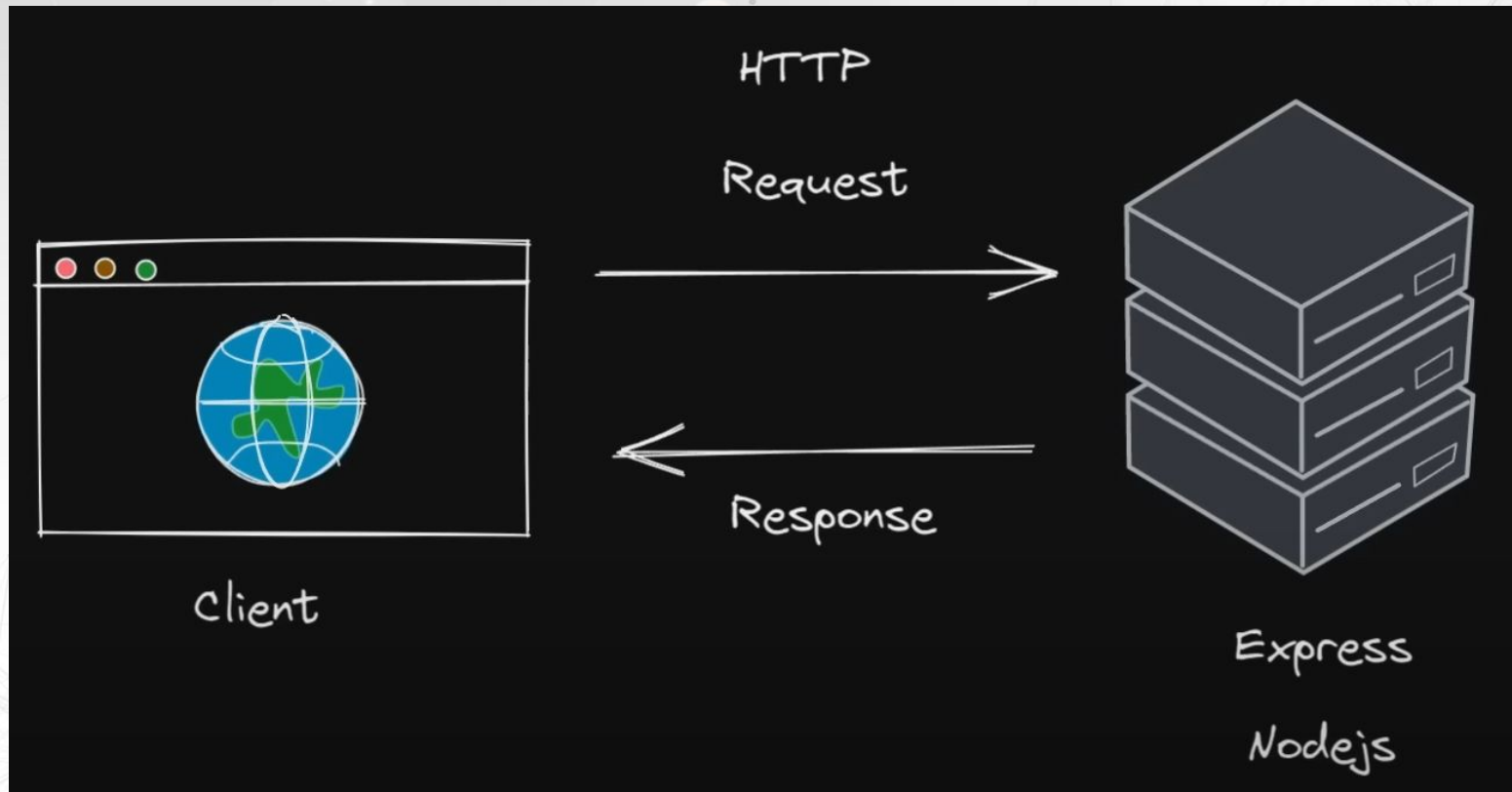
Tareas comunes de desarrollo web no están directamente soportadas por el mismo Node. Si quieres añadir el manejo específico de diferentes verbos HTTP (ej, GET, POST, DELETE, etc.), gestionar de forma separada las peticiones por medio de diferentes direcciones URL ("rutas"), servir ficheros estáticos o usar plantillas para crear la respuesta de forma dinámica, necesitarás escribir mucho código.

Express proporciona mecanismos para:

- Escritura de manejadores de peticiones con diferentes verbos HTTP en diferentes caminos URL (rutas).
- Integración con motores de renderización de "vistas" para generar respuestas mediante la introducción de datos en plantillas.
- Establecer ajustes de aplicaciones web como qué puerto usar para conectar, y la localización de las plantillas que se utilizan para renderizar la respuesta.
- Añadir procesamiento de peticiones "middleware" adicional en cualquier punto dentro de la tubería de manejo de la petición.



Comportamiento



Instalar y Ejecutar Express



Instalamos:

```
npm install express
```

Nuestro package.json agrega express como dependencia.

```
"dependencies": {  
  "express": "^4.21.0"  
}
```

Guardamos en app.js y ejecutamos :

```
var express = require('express');  
var app = express();  
app.listen(8080);
```

```
C:\Users\Your Name> node app.js
```



Endpoint



Las rutas son el mecanismo mediante el cual se define cómo responderá el servidor a las solicitudes HTTP que llegan desde el cliente a un determinado destino.

Cada ruta representa un camino o dirección en la URL y está asociada distintos algoritmos que definen la lógica que se ejecutará cuando un cliente acceda a esa ruta.

Cuando una aplicación Express recibe una solicitud, el servidor examina la URL solicitada y busca una ruta que coincida con ese patrón, junto con el método HTTP adecuado (como GET, POST, PUT, DELETE). Una vez que se encuentra una coincidencia, se ejecuta la lógica correspondiente que puede devolver datos al cliente, realizar acciones en la base de datos, o cualquier otra tarea que esté definida para esa ruta.

A dicha combinación entre método y ruta se lo denomina **endpoint**,

Componentes de un Endpoint Express

```
app.METHOD(PATH, HANDLER)
```

- **app** es una instancia de express.
- **METHOD** es un método de solicitud HTTP que define el tipo de solicitud.
- **PATH** es una vía de acceso en el servidor, especifica el patrón que debe coincidir con la URL.
- **HANDLER** Es la función que se ejecutará cuando la ruta coincida. Ésta callback recibe tres parámetros: req (solicitud), res (respuesta), y next (para la gestión de middlewares).



Routing Express



Guardamos en app.js :

```
var express = require('express');
var app = express();

app.get('/', function(peticion, respuesta) {
  respuesta.send("Mi primer pagina con
express");
});

app.listen(8080);
```

```
import express from 'express';
const app = express();

app.get('/', (req, res) => {
  res.send("Mi primer página con express");
});

app.listen(8080, () => {
  console.log("Servidor escuchando en
http://localhost:8080");
});
```

Routing Express



```
app.get('/', function(req, res) {  
  res.send('Hello World!');  
});
```

Definimos una función (callback) manejadora de ruta para peticiones HTTP GET a la raíz del sitio ('/'). Dicha función toma una petición y una respuesta como argumentos. En este caso el método simplemente llama a `send()` en la respuesta para retornar la cadena "Hello World!".

Hay un número de otros métodos de respuesta para finalizar el ciclo de solicitud/respuesta, por ejemplo podrá llamar a **`res.json()`** para enviar una respuesta JSON o **`res.sendFile()`** para enviar un archivo.

Routing Express



```
import express from 'express';
const app = express();

app.get('/', (req, res) => {
  res.send("Mi primer página con express");
});

app.get('/perfil', (req, res) => {
  res.send("Este es mi perfil");
});

app.listen(8080, () => {
  console.log("Servidor escuchando en http://localhost:8080");
});
```

Express vs HTTP



```
const express = require('express')
const app = express()

app.get('/', function (req, res) {
  res.status(200).send('Mi primer pagina con
express')
})

app.get('/perfil', function (req, res) {
  res.status(200).send('Este es mi perfil')
})

app.use((req, res) => {
  res.status(404).send('Te has perdido')
})

app.listen(8080)
```

```
const http = require('node:http')

const server = http.createServer((req, res) => {
  res.setHeader('Content-Type', 'text/plain')

  if (req.url === '/' && req.method === 'GET') {
    res.statusCode = 200
    res.end('Mi primer pagina con Node.js')
  } else if (req.url === '/perfil' && req.method
=== 'GET') {
    res.statusCode = 200
    res.end('Este es mi perfil')
  } else {
    res.statusCode = 404
    res.end('Te has perdido')
  }
})

server.listen(8080)
```

Ejemplo de los distintos métodos http



```
const express = require('express');
const app = express();

app.get('/api/prueba', function (req, res) {
  res.send("Realizo un get");
});

app.post('/api/prueba', function (req, res) {
  res.send("Realizo un post");
});

app.put('/api/prueba', function (req, res) {
  res.send("Realizo un put");
});

app.delete('/api/prueba', function (req, res) {
  res.send("Realizo un delete");
});

app.listen(8080);
```

Body



Es un campo opcional en los mensajes al que llaman cuerpo ('body' en inglés), el cual lleva los **datos asociados con la petición** (como contenido de un formulario HTML), o los archivos o documentos **asociados a una respuesta** (como un JSON, una página HTML, o un archivo de audio, vídeo, etc.).

```
const express = require('express')
const app = express()

app.use(express.text());
app.use(express.json());

app.post('/perfil', function (req, res) {
  console.log(req.body)
  res.status(200).send('Perfil creado')
})

app.use((req, res) => {
  res.status(404).send('te has perdido')
})

app.listen(3000)
```

Habilitando json en express



express.json() and **express.urlencoded()** son built-in middleware funciones necesarias para soportar JSON-encoded y URL-encoded bodies.

```
const express = require('express');  
const app = express();  
  
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

Post & Put



Ambos métodos http soportan el envío de un body también llamado como payload, en el cual el servidor recibe dichos datos, pudiendo ser estos del orden de los megas o gigas.

```
const express = require('express');
const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.post('/api/persona', function(req, res) {
  console.log(req.body);
  //todo nuestro codigo por aqui
  res.send(`Se creó la persona ${req.body.nombre} ${req.body.apellido}
  ${req.body.dni}`);
});

app.listen(8080, () => {
  console.debug('App escuchando puerto :8080');
});
```

End vs Send



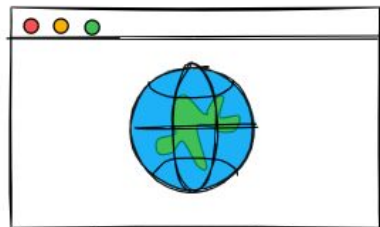
Ambos métodos se utilizan para finalizar una respuesta HTTP pero tienen características diferentes

- **res.end():** es nativo del módulo 'http' en Node.js, no incluye ningún cuerpo de respuesta (aunque puedes enviar un cuerpo si lo incluyes como argumento opcional) y no establece automáticamente el tipo de contenido ni hace ninguna transformación de datos.
- **res.send():** es nativo de Express, permite enviar diferentes tipos de datos (texto, objetos, arrays, html, json) y automáticamente realiza algunas tareas adicionales, como detectar el tipo de contenido y establecer el encabezado Content-Type correctamente, así como convertir automáticamente los objetos o arrays a JSON.

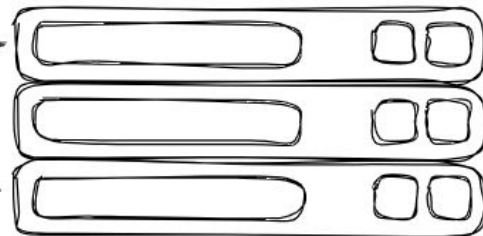
Requests en Express

Expandiendo el protocolo HTTP

URL → localhost:8080/usser/create
Method → POST
Headers → accept:text/html
Body → {"name" = "pepe", "age" = 30}



HTTP Request



HTTP Response

endpoint → /usser/create
Method → POST
Headers → (200) + JSON
Body → {error, menssaje}

Usando partes de la URL

Usando path como parámetros



```
const express = require('express');
const app = express();
app.get('/user/:id', function(req, res) {
  res.send(req.params);
});

app.listen(8080, () => {
  console.debug('App escuchando puerto :8080');
});
```



Usando partes de la URL

Colisión en definición de URL

```
const express = require('express');
const app = express();

app.get('/user/:id', function(req, res) {
  res.send(req.params);
});

app.get('/user/perfil', function(req, res) {
  res.send("Perfil de cuenta");
});

app.listen(8080, () => {
  console.debug('App escuchando puerto :8080');
});
```

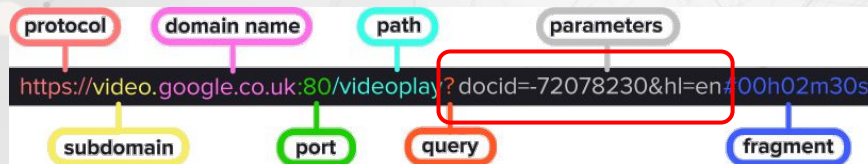
Que se
ejecutará?

<http://localhost:8080/user/perfil>



Usando partes de la URL

Obteniendo los query params



```
const express = require('express');
const app = express();
app.get('/api', function(req, res) {
  res.send(JSON.stringify(req.query));
});
app.listen(8080, () => {
  console.debug('App escuchando puerto :8080');
});
```

`http://localhost:8080/api?nombre=jose&apellido=perez`



Usando partes de la URL

Podemos utilizarlo de forma combinada

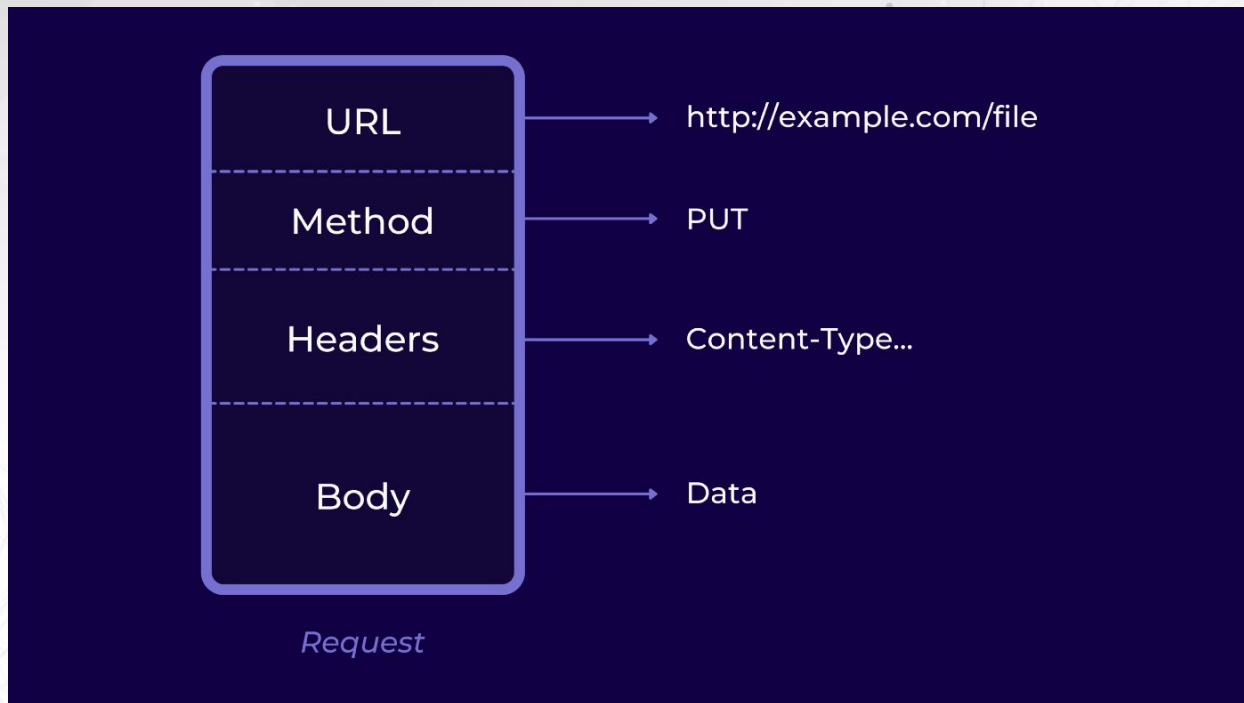


```
const express = require('express');
const app = express();
app.get('/api/:year', function(req, res) {
  res.send(JSON.stringify(req.query) + JSON.stringify(req.params));
});
app.listen(8080, () => {
  console.debug('App escuchando puerto :8080');
});
```

`http://localhost:8080/api/2024?nombre=jose&apellido=perez`



Formato a alto nivel



Formato a alto nivel

```
POST / HTTP/1.1
```

```
Host: localhost:8000
```

```
User-Agent: Mozilla/5.0 (Macintosh;... )... Firefox/51.0
```

```
Accept: text/html,application/xhtml+xml,..., */*;q=0.8
```

```
Accept-Language: en-US,en;q=0.5
```

```
Accept-Encoding: gzip, deflate
```

```
Connection: keep-alive
```

```
Upgrade-Insecure-Requests: 1
```

```
Content-Type: multipart/form-data; boundary=-12656974
```

```
Content-Length: 345
```

```
-12656974
```

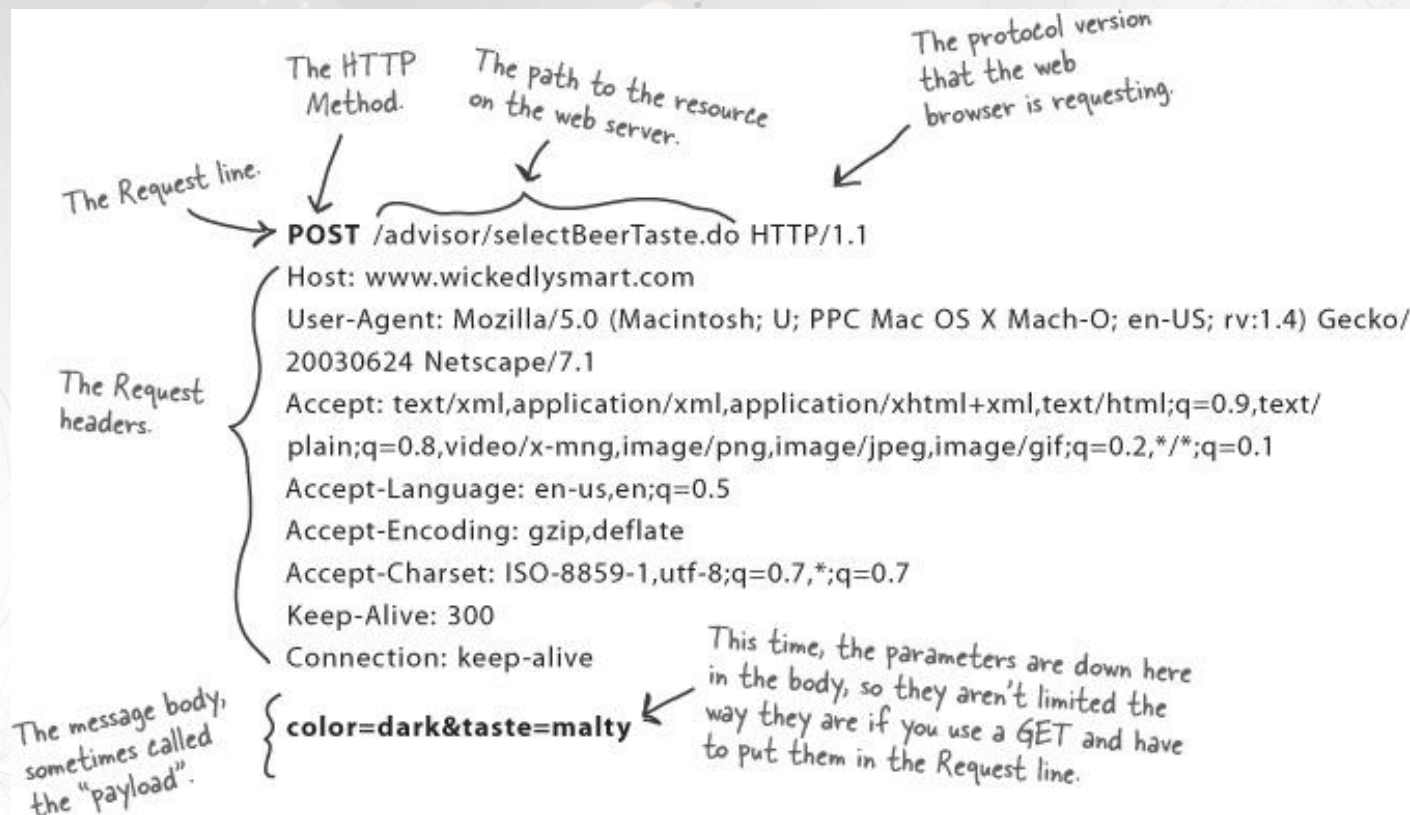
```
(more data)
```

Request headers

General headers

Representation
headers

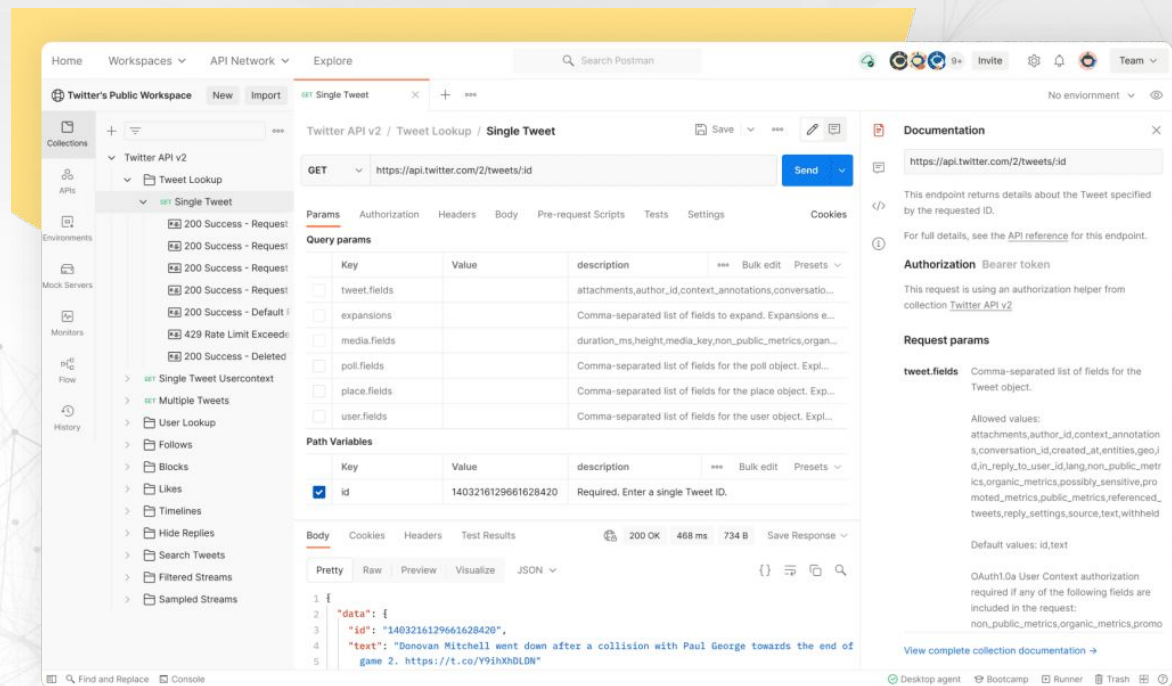
Formato a alto nivel



Postman

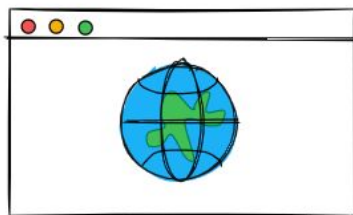


Para poder realizar request con métodos POST/PUT/DELETE/... necesitamos un software que nos permita realizar dichas operaciones, recomendamos instalar postman



Analizando los endpoints

URL → localhost:8080/usser/create
Method → POST
Headers → accept:text/html
Body → {"name" = "pepe", "age" = 30}



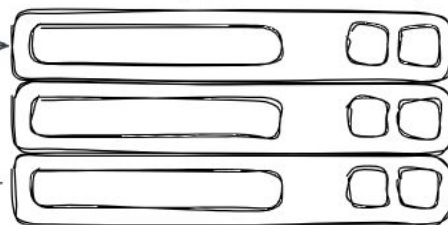
HTTP Request

HTTP Response

endpoint → /usser/create
Method → POST
Headers → (200) + JSON
Body → {"name" = "pepe", "age" = 30}

```
app.post('/api/persona', (req, res) => {  
  "httpVersion": req.httpVersion,  
  "headers": req.headers,  
  "url": req.url,  
  "method": req.method,  
  "query": req.query,  
  "params": req.params  
  "body": req.body  
});
```

Express → app.METHOD("/path", handler)



```
app.post('/api/persona', (req, res) => {  
  res.end('string');  
  res.send('string');  
  res.sendFile('/file.png');  
  res.download('/file.png');  
  res.json({"key": "value"});  
  res.status(###).send('string');  
  res.sendStatus(###);  
});
```

express.Router



Es una característica que funciona como una mini-aplicación dentro de Express que te permite modularizar y organizar mejor las rutas de tu aplicación de forma independiente.

Para mejorar la organización de proyectos grandes es recomendable separar las rutas en diferentes archivos. También puedes utilizar middlewares locales si necesitas aplicar lógica sólo a ciertas rutas.

/raiz

- | — /config → Configuración de la conexión a la base de datos
- | — /controllers → Controladores que manejan la lógica de distintos entornos
- | — /middlewares → Por ejemplo un middleware para autenticación
- | — /models → Modelos que gestionan los datos desde las fuentes para abastecer a los controladores
- | — /routes → Rutas que derivan el tráfico a los distintos controladores
- | — /services → Lógica adicional para interactuar con la base de datos
- | — /utils → por ejemplo un errorHandler.js (Manejador de errores genérico)
- | — app.js → Punto de entrada de la aplicación
- | — package.json → Dependencias y scripts del proyecto

Middleware

Mientras las funciones de enrutamiento, con el objeto **express.Router**, se encargan del ciclo petición-respuesta, al gestionar la respuesta adecuada al cliente, las funciones de middleware normalmente realizan alguna operación al gestionar una petición o respuesta y a continuación llaman a la siguiente función en la "pila", que puede ser otra función de middleware u otra función de enrutamiento. El orden en el que las funciones de middleware son llamadas depende del desarrollador de la aplicación.

La mayoría de las aplicaciones usan middleware desarrollado por terceras partes, para simplificar funciones habituales en el desarrollo web, como puede ser: gestión de cookies, sesiones, autenticado de usuarios, peticiones POST y datos en JSON, registros de eventos, etc



Middleware



En Express, un middleware es una función que tiene acceso al objeto request (req), al objeto response (res) y a una función next() que se utiliza para pasar el control al siguiente middleware o controlador.

```
function midd(req, res, next) {  
  // Lógica de manipulación o validación  
  
  // finaliza la respuesta o llama a `next()` para continuar  
  next();  
}
```

Express permite pasar una serie de middlewares antes de la función de manejo de la ruta (controlador). Cuando haces esto, los middlewares se ejecutan de manera secuencial, es decir, uno tras otro en el orden en que son definidos.

```
router.get('/ruta', middleware1, middleware2, controlador);
```

Imprimiendo datos del request

```
const express = require('express');
const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use(function(req, res, next) {
  var data = {
    "httpVersion": req.httpVersion,
    "headers": req.headers,
    "url": req.url,
    "method": req.method,
    "query": req.query
  };
  console.log(JSON.stringify(data));
  next();
});

app.post('/api/persona', function(req, res) {
  console.log(req.body);
  //todo nuestro codigo por aqui
  res.send(`Se creó la persona ${req.body.nombre} ${req.body.apellido} ${req.body.dni}`);
});

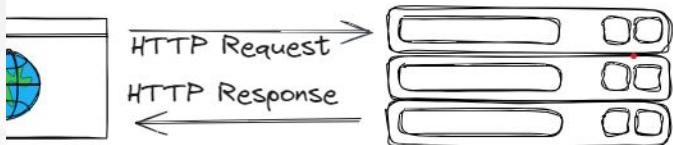
app.listen(8080);
```

Capturamos todo el tráfico e imprimimos lo que recibimos

Trabajando con endpoints

```
app.post('/api/persona', (req, res) => {  
  "httpVersion": req.httpVersion,  
  "headers": req.headers,  
  "url": req.url,  
  "method": req.method,  
  "query": req.query,  
  "params": req.params  
  "body": req.body  
});
```

Express → app.METHOD("/path", handler)



```
app.post('/api/persona', (req, res) => {  
  res.end('string');  
  res.send('string');  
  res.sendFile('/file.png');  
  res.download('/file.png');  
  res.json({"key": "value"});  
  res.status(###).send('string');  
  res.sendStatus(###);  
});
```

Middleware

```
app.use(morgan('tiny'));  
app.param('param', (req, res, next, param) => {  
  next();  
});
```

Controllers

```
usser.create((req, res) => {  
  usserDB.create(req.params, (err, result) => {  
    ...  
  });  
});  
  
usser.delete((req, res) => {  
  usserDB.delete(req.params, (err, result) => {  
    ...  
  });  
});
```

DataBase

```
var connection = mysql.createConnection({ ... });  
usserDB.create(req.params, (err, result) => {  
  query = "INSERT INTO USSER VALUES ..."  
  connection.query($query, (err, rows, fields) => {  
    if(err){  
      ... return database err;  
    }else{  
      ... return database rows;  
    }  
  });  
});  
connection.end();
```

Usando partes de la URL



Middleware transformando parámetros

```
app.param("name", function (req, res, next, name) {  
  const modified = name.toUpperCase();  
  req.name = modified;  
  next();  
});  
  
app.get("/aviones/:name", function (req, res) {  
  res.send("Avion buscado: " + req.name);  
});
```

<http://localhost:8080/aviones/figther>

Usando partes de la URL



Middleware validando datos

```
const express = require('express');
const app = express();

app.param('id', function (req, res, next, name) {
  if (!isNaN(id) && !isNaN(parseInt(id))) {
    next();
  } else {
    res.send("ID debe ser un número entero")
  }
});

app.get('/users/:id', function (req, res) {
  res.send('Hola ID' + req.params.id);
});

app.listen(8080, () => {
  console.debug('App escuchando puerto :8080');
});
```

Morgan logging tool (middleware)



Es una gran herramienta que registra las requests junto con alguna otra información dependiendo de su configuración y el valor predeterminado utilizado. Demuestra ser muy útil durante la depuración y también si desea crear archivos de registro.

```
const express = require('express');
const morgan = require('morgan');

const app = express();
app.use(morgan('tiny'));
morgan(':method :url :status :res[content-length] - :response-time ms');

app.listen(3000, () => {
  console.debug('App listening on :3000');
});

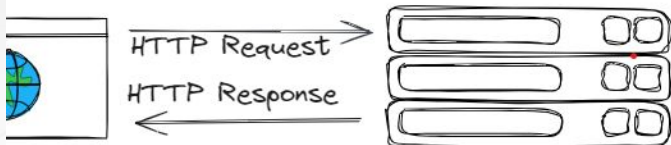
app.get("/", (req, res) => {
  res.send("Prueba log");
});
```

```
C:\Users\Your Name> npm install morgan
```

Trabajando con endpoints

```
app.post('/api/persona', (req, res) => {  
  "httpVersion": req.httpVersion,  
  "headers": req.headers,  
  "url": req.url,  
  "method": req.method,  
  "query": req.query,  
  "params": req.params  
  "body": req.body  
});
```

Express → app.METHOD("/path", handler)



```
app.post('/api/persona', (req, res) => {  
  res.end('string');  
  res.send('string');  
  res.sendFile('/file.png');  
  res.download('/file.png');  
  res.json({"key": "value"});  
  res.status(###).send('string');  
  res.sendStatus(###);  
});
```

Middleware

```
app.use(morgan('tiny'));  
app.param('param', (req, res, next, param) => {  
  next();  
});
```

Controllers

```
usser.create((req, res) => {  
  usserDB.create(req.params, (err, result) => {  
    ...  
  });  
});  
  
usser.delete((req, res) => {  
  usserDB.delete(req.params, (err, result) => {  
    ...  
  });  
});
```

DataBase

```
var connection = mysql.createConnection({ ... });  
usserDB.create(req.params, (err, result) => {  
  query = "INSERT INTO USSER VALUES ... "  
  connection.query($query, (err, rows, fields) => {  
    if(err){  
      ... return database err;  
    }else{  
      ... return database rows;  
    }  
  });  
});  
connection.end();
```

Conectándonos a una DB



Primero es necesario instalar el paquete de mysql

```
var mysql = require('mysql');

// Agregue las credenciales para acceder a su base de datos
var connection = mysql.createConnection({
  host      : 'localhost',
  user      : '<USERNAME that typically is root>',
  password  : '<PASSWORD or just use null if youre working locally>',
  database  : '<DATABASE-NAME>'
});

connection.connect(function(err) { // conectarse a mysql
  if(err){ // en caso de error
    console.log(`${err.code} ${err.fatal}`);
  }
});
```

```
C:\Users\Your Name> npm install mysql
```

Query a una DB

```
var mysql = require('mysql');
var connection = mysql.createConnection({
  host      : 'localhost',
  user      : 'root',
  password  : 'root',
  database  : 'silicon'
});

connection.connect(function(err) {
  if(err){
    console.log(err);
  }else{
    console.log("Conexion a DB exitosa");
  }
});

query = 'SELECT * from alumnos';
connection.query(query, function(err, rows, fields) {
  if(err){
    console.log("An error ocurred performing the query.");
    return;
  }
  console.log("Consulta ejecutada con éxito:", rows);
});
connection.end();
```



Query con parametros

```
var id = 5;
connection.query('DELETE from alumnos WHERE alumno_id=?', id,
function (err, rows, fields) {
    if (err) {
        console.log("An error ocurred performing the query.");
        return;
    }
    console.log("Consulta ejecutada con éxito:", rows);
}));
```

Se usa el comodín “?” para indicar que se recibirá un parámetro como variable, las mismas se envían a la función query, luego del string de la sentencia SQL.



Promise

Una Promise (promesa en castellano) es un objeto que representa la terminación o el fracaso de una operación asíncrona. Esencialmente, una promesa es un objeto devuelto al cual se adjuntan funciones callback, en lugar de pasar callbacks a una función.

```
function enCasoDeExito(resultado) {  
    console.log("Archivo de audio disponible en la URL " + resultado);  
}  
  
function enCasoDeFracaso(error) {  
    console.log("Error generando archivo de audio " + error);  
}  
  
crearArchivoAudioAsync(audioConfig, enCasoDeExito, enCasoDeFracaso);
```



```
const promesa = crearArchivoAudioAsync(audioConfig);  
promesa.then(enCasoDeExito, enCasoDeFracaso);
```



Promise



Se puede ver como un valor que no necesariamente se conoce cuando se crea la promesa. Esto permite que los métodos asíncronos devuelvan la promesa de proporcionar el valor en algún momento en el futuro, en lugar de devolver inmediatamente el valor final.

- **pending (pendiente)**: estado inicial, ni cumplido ni rechazado.
- **fulfilled (cumplida)**: lo que significa que la operación se completó con éxito.
- **rejected (rechazada)**: lo que significa que la operación falló.

Una promesa pendiente puede cumplirse con un valor o rechazarse con un motivo (error). Cuando ocurre cualquiera de estas opciones, se llama a los controladores asociados en cola por el método **then** de una promesa.

Una promesa se crea utilizando el constructor **Promise**. Este constructor acepta una función callback que recibe dos parámetros: **resolve** y **reject**. Estas funciones se utilizan para cambiar el estado de la promesa.

Promise



Para manejar el resultado de una promesa, se utilizan los métodos `.then()` y `.catch()`.

`then()`: Se ejecuta cuando la promesa se resuelve con éxito (estado fulfilled).

`catch()`: Se ejecuta cuando la promesa es rechazada (estado rejected).

```
let promesa = new Promise((resolve, reject) => {  
  // simulamos la asincronía con setTimeout  
  setTimeout(() => {  
    let exito = true;  
    if (exito) {  
      resolve("¡Operación exitosa!");  
    } else {  
      reject("Error en la operación");  
    }  
  }, 2000);  
});
```

```
promesa.then((resultado) => {  
  console.log(resultado);  
}).catch((error) => {  
  console.log(error);  
});
```

async/await



Es una forma más moderna y legible de trabajar con promesas. Simplifica el código al permitir escribir operaciones asíncronas de una manera que parece sincrónica.

async: Se coloca antes de una función para indicar que la función esperará una promesa.

await: Se utiliza dentro de una función async para esperar el resultado de una promesa. Todo lo que retorne dentro de la función será automáticamente envuelto en una promesa. Detiene la ejecución de la función hasta que la promesa se resuelva o se rechace.

```
async function operacionAsincrona() {  
  try {  
    let resultado = await promesa;  
    console.log(resultado);  
  } catch (error) {  
    console.log(error);  
  }  
}  
  
operacionAsincrona();
```

¿Qué cambia?



- Cuando definimos que el **return** de una función será una nueva **Promise**, estamos estableciendo que ahora devuelve una promesa en lugar de ejecutar un callback. Esta promesa puede venir resuelta (resolve) o rechazada (reject).
- Una función definida como asíncrona utilizando **async**, será aquella que reciba las promesas utilizando **await** para esperar a que se resuelva o se rechace la promesa retornada.
- Ya no utilizamos **.then().catch()** sino que englobamos los renglones en un bloque **try...catch** para manejar el éxito o fracaso de la promesa.
- En **async/await**, la función que se invoca tras escribir **await** debe devolver una promesa. Esto se debe a que el propósito de **await** es "esperar" a que una promesa sea resuelta o rechazada. Lo que realmente está ocurriendo es que el código se "**pausa**" hasta que la promesa se complete. Si la función no devuelve una promesa, **await** no tiene algo a lo que esperar. Sin embargo, **JS automáticamente convertirá el valor retornado en una promesa si no lo es**. Esto significa que si la función devuelve un valor normal (no una promesa), **await lo envolverá en una promesa resuelta**.

¿Debo utilizar promesas?

En este caso, `funcionNormal()` devuelve un valor normal, pero `await` lo envuelve implícitamente en una promesa resuelta (`resolve`). El código sigue funcionando, pero no es necesario esperar, ya que el valor es retornado inmediatamente.

En el **caso ideal**, la función tras `await` **debería devolver una promesa**, ya que el objetivo de `await` es justamente esperar su resolución. Pero no es obligatorio, ya que si la función devuelve un valor normal, `await` lo maneja como si fuera una promesa resuelta inmediatamente.

```
function funcionNormal() {  
    return "Valor inmediato";  
}  
  
async function usarFuncionNormal() {  
    const resultado = await funcionNormal();  
    console.log(resultado);  
}  
  
usarFuncionNormal();
```

.then() vs await



```
fetch(url, request).then(response => {
  try {
    console.log(response);
    return response.json().
      then(data => ({
        status: response.status,
        ok: response.ok,
        headers: response.headers,
        data: data
      }))
  }
});
} catch (error) {
  console.log(error);
}
}).then(json => {
  console.log(json);
}).catch(error => {
  console.log(error);
});
```

```
async function buscarDatos(url, request) {
  try {
    const response = await fetch(url, request);
    console.log(response);
    const data = await response.json();

    const result = {
      status: response.status,
      ok: response.ok,
      headers: response.headers,
      data: data
    };
    console.log(result);
    return result;
  } catch (error) {
    console.log(error);
  }
}
buscarDatos(url, request);
```

Callbacks vs Async/Await



```
function buscarLatita(responsable, callback) {
  console.log( responsable + " está revisando la heladera..." );
  setTimeout(() => {
    habia = true;
    if (habia) {
      callback(null, "una gaseosa helada");
    } else {
      callback("no había gaseosa fría", null);
    }
  }, 2000);
}

function verQueHacer(err, result){
  if (err) {
    console.warn("Tendré que esperar porque" , err);
  } else {
    console.info("Genial, gracias hijo por traer" , result);
  }
}

buscarLatita('pedrito', verQueHacer);
```

Callbacks vs Async/Await



```
function buscarLatita(responsable) {
  console.log(responsable + " está revisando la heladera...");
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let habia = false;
      if (habia) {
        resolve("una gaseosa helada"); // Resolvemos la promesa con éxito
      } else {
        reject("no habia gaseosa fría"); // Rechazamos la promesa por err
      }
    }, 2000);
  });
}

async function verQueHacer() {
  try {
    const exito = await buscarLatita('pedrito');
    console.info("Genial, gracias hijo por traer" , exito);
  } catch (err) {
    console.warn("Tendré que esperar porque" , err);
  }
}

verQueHacer();
```

¿Qué cambia?



- **buscarLatita()** ahora devuelve una promesa en lugar de usar un callback. Si se encuentra la latita, la promesa se resuelve (resolve), y si no, se rechaza (reject).
- **verQueHacer()** se convierte en una función asíncrona utilizando async. Dentro de ella, se utiliza await para esperar a que se resuelva o se rechace la promesa retornada por buscarLatita.
- Utilizamos un bloque **try...catch** en verQueHacer para manejar el éxito o fracaso de la promesa, reemplazando el callback.
- En async/await, la función que se invoca tras escribir await **debe devolver una promesa**. Esto se debe a que el propósito de await es "esperar" a que una promesa sea resuelta o rechazada.
 - Lo que realmente está ocurriendo es que el código se "**pausa**" hasta que la promesa se complete (resuelva o rechace). Si la función no devuelve una promesa, await no tiene algo a lo que esperar. Sin embargo, JS automáticamente **convertirá el valor retornado en una promesa si no lo es**. Esto significa que si la función devuelve un valor normal (no una promesa), await lo envolverá en una promesa resuelta.

ApiCall (Consultando datos remotos)



Las **llamadas a API** se pueden realizar tanto en proyectos de frontend (como en React) como en backend (con Node.js y Express) y son realizadas tanto con librerías nativas como con módulos de terceros, entre ellos están **fetch**, **axios**, **jQuery AJAX** u otras librerías similares

En el frontend, las llamadas a API (fetch o axios) se suelen hacer para obtener datos desde un servidor externo, generalmente una API REST. Ejemplos comunes incluyen obtener datos para mostrar en el sitio o enviar información del usuario al backend.

En el backend, las solicitudes a API se hacen comúnmente cuando el servidor necesita interactuar con otros servicios o APIs externas y permiten, por ejemplo, obtener datos de una API externa y luego enviarlos al frontend



Fetch

La Fetch API es una API nativa de JavaScript para hacer peticiones HTTP. Aunque no requiere instalación, es más básica que Axios, pero es lo suficientemente flexible para la mayoría de las aplicaciones.

Provee un método global `fetch()` que proporciona una forma fácil y lógica de obtener recursos de forma **asíncrona** por la red. Este tipo de funcionalidad se conseguía previamente haciendo uso de **XMLHttpRequest**.

En Node.js, puedes utilizar `node-fetch` para implementar `fetch` en el backend, ya que `fetch` no está disponible de manera nativa en versiones anteriores de Node.js.

`npm install node-fetch`

```
fetch('http://example.com/movies.json')  
  .then(response => response.json())  
  .then(data => console.log(data));
```



node-fetch

Tener en cuenta que solamente funciona con ES Modules.

```
                                ASYNC AWAIT
app.get("/api/users", async (req, res) => {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users" );
    const data = await response.json();
    res.json(data);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Error en la petición" });
  }
});

                                PROMISE
app.get("/api/users", (req, res) => {
  fetch("https://jsonplaceholder.typicode.com/users" )
    .then(response => response.json())
    .then(data => res.json(data))
    .catch(error => {
      console.error(error);
      res.status(500).json({ message: "Error en la petición" });
    });
});
```



Model - Controller

```
app.get('/api/persona/', function (req, res) {
  connection.query("SELECT * FROM personas", function (err, result, fields) {
    if (err) {
      res.status(500).send({
        message: "Surgio un problema, contactese con un administrador. Gracias",
        detail: err
      });
      console.error(err);
    } else {
      res.json(result);
    }
  });
});
```

```
app.get('/api/persona/', function (req, res) {
  personaDb.getAll(function (err, result) {
    if (err) {
      res.status(500).send(err);
    } else {
      res.json(result);
    }
  });
});
```

```
var personaDb = {};

personaDb.getAll = function(funCallback){
  connection.query("SELECT * FROM personas", function (err, result, fields) {
    if (err) {
      funCallback({
        message: "Surgio un problema, contactese con un administrador. Gracias",
        detail: err
      });
      console.error(err);
    } else {
      funCallback(undefined, result);
    }
  });
}
```

Response en Express

response.append

Usado para agregar HEADERS a la respuesta

```
const express = require('express');
const app = express();
app.post('/api/prueba', function (req, res) {
  res.append('Link', ['<http://localhost/>',
    '<http://localhost:3000/>'])
  res.append('Set-Cookie', 'foo=bar; Path=/; HttpOnly')
  res.append('Warning', '199 Miscellaneous warning')
  res.send("Realizo un post");
});
app.listen(8080);
```



response.attachment

Usado para que la respuesta se descargue como un archivo

```
const express = require('express');
const app = express();

app.get('/api/prueba', function (req, res) {
  res.attachment('archivo.txt');
  res.end("Este es el contenido");
});

app.listen(8080);
```



response.attachment

Usado para que la respuesta se descargue como un archivo

```
const express = require('express');
const app = express();
var fs = require('fs');

app.get('/api/prueba', function (req, res) {
  //deben tener una imagen con dicho nombre en la carpeta del proyecto
  var imagen = fs.readFileSync(__dirname + '/imagen.jpeg');
  res.attachment('imagen.jpeg')
  res.end(imagen);
});

app.listen(8080);
```



response.download

Usado para descargar un archivo

```
const express = require('express');
const app = express();
var fs = require('fs');

app.get('/api/prueba', function (req, res) {
  res.download('./imagen.jpeg', "imagen2.jpeg")
});

app.listen(8080);
```



Respondiendo archivos estáticos

Usado leer de una carpeta archivos, util para usar en front-end.
En el código ejemplo, todo lo que se encuentre en la carpeta “public” dentro de mi proyecto, va a ser accesible desde la URL “static” de mi sitio web

```
const express = require('express');  
const app = express();  
  
app.use('/static', express.static('public'))  
  
app.listen(8080);
```



Encriptando password

Una buena práctica siempre conlleva utilizar algún mecanismo para encriptar las passwords antes de almacenarlas en la DB. para ello usaremos:

<https://www.npmjs.com/package/bcrypt>

Encriptando:

```
const bcrypt= require('bcrypt');  
let textoHashed = bcrypt.hashSync("texto a encriptar",10);  
console.log(textoHashed)
```

Comparando:

```
const bcrypt= require('bcrypt');  
const iguales = bcrypt.compareSync("texto plano", "texto encriptado");  
console.log(iguales)
```



Encriptando password



Para hacer pruebas de bcrypt por ejemplo para testear valores cifrados o para generar valores por ejemplo para un proceso de ingesta de datos mediante SQL donde no tenemos posibilidad de usar node, podemos utilizar:
<https://bcrypt-generator.com/>

Es una web que nos permite generar/chequear los encriptados.



JsonWebToken

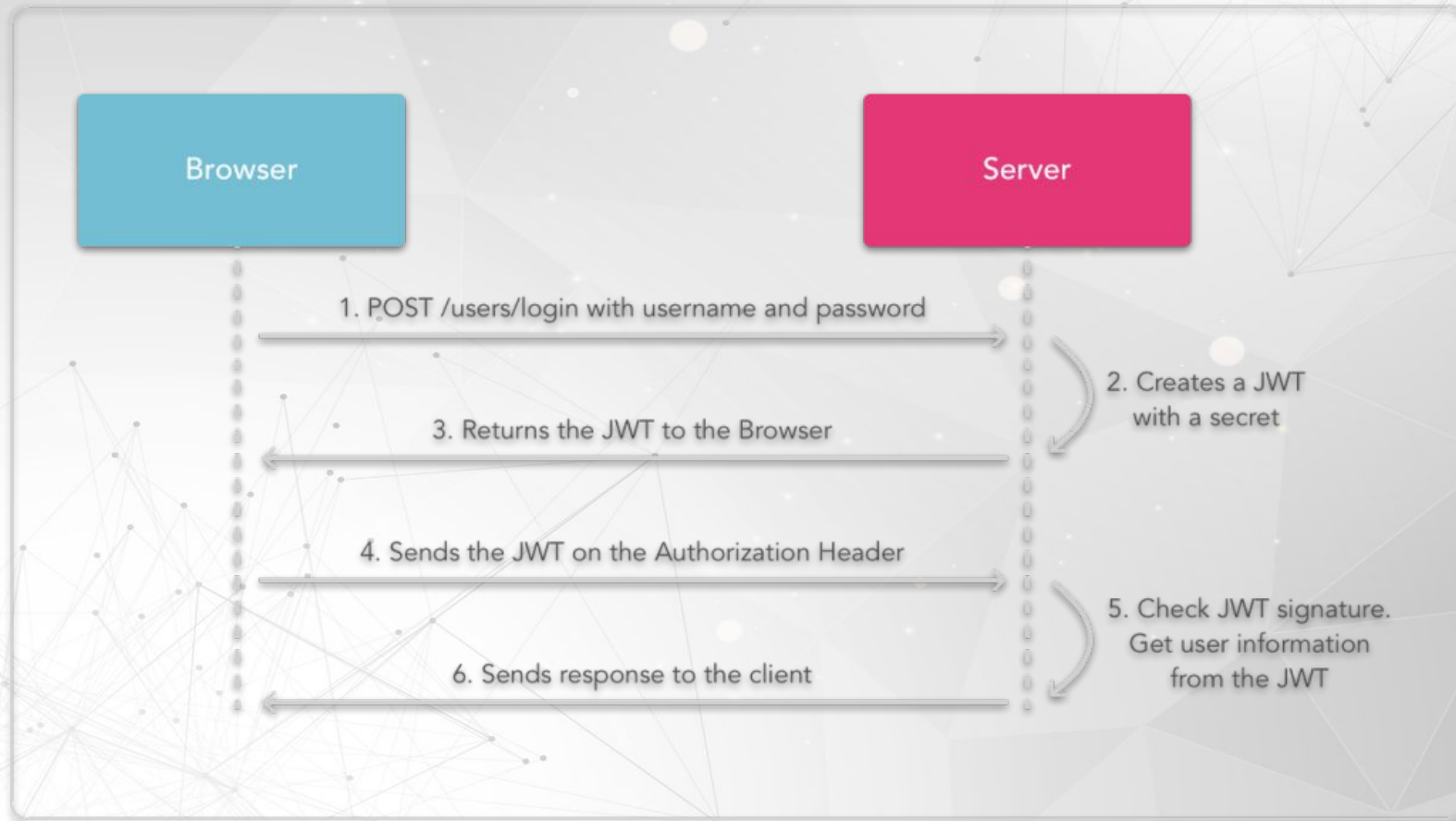


JWT (JSON Web Token) es un estándar que está dentro del documento RFC 7519.

En el mismo se define un mecanismo para poder propagar entre dos partes, y de forma segura, la **identidad** de un determinado usuario, además con una serie de claims o **privilegios**.

Estos privilegios están codificados en objetos de tipo JSON, que se incrustan dentro de del payload o cuerpo de un mensaje que va firmado digitalmente.

JsonWebToken



Home Page



Generando:

```
const jwt = require('jsonwebtoken');
jwt.sign({ "data": "misdatostoken" }, 'clavesecreta', { expiresIn: '1200s' }, (err, token) => {
  if (!err) {
    console.log(token);
  } else {
    console.log(err);
  }
})
```

Verificando:

```
const jwt = require('jsonwebtoken');
try {
  const token = "recibido en header de request"
  const verified = jwt.verify(token, "clavesecreta");
  console.log(verified);
} catch (error) {
  console.log('token no es válido')
}
```

Validación de datos

Validación clásica

Las validaciones en Express son un conjunto de funciones que permiten validar y depurar los datos de entrada de los usuarios en aplicaciones en línea.

```
function validateUser(req, res, next) {  
  const { username, age } = req.body;  
  if (!username || typeof username !== 'string') {  
    return res.status(400).send('Username is required and should be a string');  
  }  
  if (age && typeof age !== 'number') {  
    return res.status(400).send('Age should be a number');  
  }  
  next();  
}  
  
app.post('/api/users', validateUser, (req, res) => {  
  // Datos válidos, procesar la solicitud  
  res.send('Usuario creado');  
});
```



validator.js



El módulo Validator de Express es un conjunto completo de funciones de validación y sanitización que los desarrolladores pueden incorporar rápidamente en sus controladores y rutas. Está construido sobre la arquitectura middleware de Express.js. Los desarrolladores pueden establecer condiciones como campos obligatorios, tipos de datos, longitudes, formatos.

express-validator es un conjunto de middlewares de express.js que envuelve la extensa colección de validadores y sanitizadores que ofrece validator.js

La idea principal es ubicar el middleware de validación en las rutas o en los controladores para verificar que los datos enviados por los clientes cumplan con los criterios que definas.

```
npm install express-validator
```



express-validator

- **Selectores de valores:**

- `check('campo')` o `body('campo')`: Verifica los datos dentro de un campo específico, con ello validamos su existencia y sus tipos de datos a validar (email, longitud, números, etc.).
- `query('campo')`: Para validar parámetros en la query string (URL).
- `param('campo')`: Para validar parámetros de rutas.

- **Funciones de validación:** son aquellas que realizan una comprobación específica pero no modifican la entrada original. (`notEmpty()`, `isEmail()`, `isNumeric()`, `optional()`, etc).

- **Funciones de sanitización:** conjunto de funciones que se utilizan para limpiar y formatear los datos de entrada antes de la validación. (`normalizeEmail()`, `trim()`, `toInt()`, `toLowerCase()`, `normalizeEmail()`. etc)



express-validator

El método `validationResult()` recolecta los errores de validación acumulados y permite manejar la respuesta si se encuentran problemas.

Se utiliza después de definir las reglas de validación (como las que creamos con `check` o `body`) y sirve para verificar si la solicitud pasó todas las validaciones o si se detectaron errores. **Devuelve un objeto Result** que contiene los errores. Si no hay errores, el resultado estará vacío.

Flujo de validationResult:

- Primero, definimos las reglas de validación con `check`, `body`, `param`, etc.
- Cuando llega una solicitud HTTP, `express-validator` ejecuta las reglas.
- Recolectamos los errores usando `validationResult(req)`.
- Si hay errores, enviamos una respuesta con ellos; si no, continuamos con el flujo (`next()`).



express-validator



body: Se utiliza específicamente para validar valores que vienen en el cuerpo de la solicitud (req.body). Solo se enfoca en esta fuente y se usa cuando tienes la certeza de que los datos vendrán en req.body.

check: Es más genérico y puede usarse para validar campos desde múltiples lugares, incluyendo req.body, req.query, req.params, e incluso req.cookies. Puede ser útil cuando necesitas validar un campo pero no sabes de antemano si vendrá en el cuerpo, en los parámetros de la URL o en la query de la URL.

Funciones de validación (algunas)



```
body('nombre').exists().withMessage('Debe proporcionar un nombre')
body('nombre').isLength({ min: 3, max: 50 }).withMessage('El nombre debe tener entre 3 y 50 caracteres')
body('nombre').notEmpty().withMessage('El nombre es obligatorio')
body('nombre').isAlpha().withMessage('El nombre solo debe contener letras')

body('email').isEmail().withMessage('Debe proporcionar un email válido')
body('edad').isInt({ min: 1, max: 120 }).withMessage('Debe proporcionar una edad válida')
body('precio').isNumeric().withMessage('Debe proporcionar un valor numérico')
body('precio').isDecimal().withMessage('Debe proporcionar un número decimal')
body('peso').isFloat({ min: 0.1 }).withMessage('Debe proporcionar un peso válido')
body('activo').isBoolean().withMessage('Debe ser un valor booleano')
body('fecha_nacimiento').isDate().withMessage('Debe proporcionar una fecha válida')
body('username').isAlphanumeric().withMessage('El nombre de usuario solo debe contener letras y números')
body('id').isUUID(4).withMessage('Debe proporcionar un UUID válido')
body('website').isURL().withMessage('Debe proporcionar una URL válida')
body('telefono').isMobilePhone('es-ES').withMessage('Debe proporcionar un número de teléfono válido')
body('tarjeta').isCreditCard().withMessage('Debe proporcionar un número de tarjeta válido')

body('nickname').optional().isLength({ max: 30 }).withMessage('El apodo no puede exceder los 30 caracteres')
body('tipo').isIn(['admin', 'user']).withMessage('El tipo debe ser "admin" o "user"')
body('password').matches(/\d/).withMessage('La contraseña debe contener al menos un número')
body('password').isStrongPassword().withMessage('La contraseña debe ser fuerte')
```

Flujo de datos



Flujo de datos entre routers, controllers y models para mantener una estructura modular y separada, escalable y relativamente fácil de mantener

- Request: El cliente envía una solicitud POST a `/api/users/register`.
- Router: La ruta aplica las reglas de validación y llama al controlador.
- Middleware: `express-validator` valida los datos según las reglas definidas.
- Controller: Si los datos son válidos, el controlador maneja la lógica del negocio.
- Model: El modelo interactúa con la base de datos para crear un nuevo usuario.
- Response: El controlador envía una respuesta al cliente.

Rutas back vs. front



Rutas en Backend (Express)	Rutas en Frontend (React Router DOM)
Definir cómo el servidor responde a las solicitudes HTTP (gestión de datos, respuestas).	Definir qué componentes o vistas se muestran según la URL del navegador.
Involucra diferentes métodos HTTP	Sólo maneja la URL y la navegación del cliente.
Puede acceder a bases de datos, archivos, etc.	Maneja la interfaz de usuario y el estado de la aplicación.
Requiere una solicitud del cliente para obtener una respuesta.	La navegación es inmediata, cambiando de componente en el cliente sin recargar la página.
Puede incluir parámetros en la URL (/usuarios/:id).	También lo hace (/usuarios/:id) pero sólo afecta el rendering de componentes.

¡Gracias!

